

실시간 Car Market 서비스

바이브 코딩 활용 매뉴얼

Firestore Auth · MongoDB Atlas · 사진 업로드 · Socket.io 상담 · Render 배포

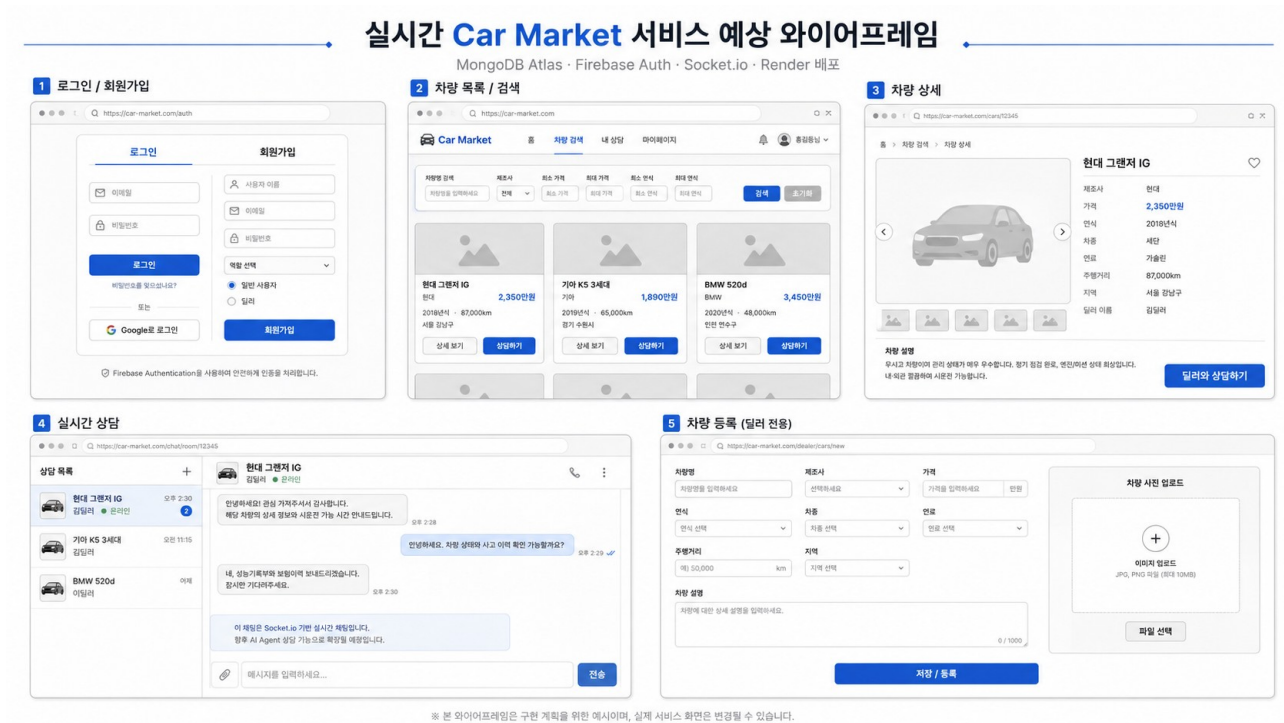


그림 1. 실시간 Car Market 서비스 예상 와이어프레임

1. 이 과제의 진행 방식

이번 과제는 모든 코드를 외워서 작성하는 과제가 아니다. 요구사항을 이해하고, AI 에게 작업 단위를 나누어 요청하면서 결과물을 완성한다.

단, AI 가 만든 코드를 그대로 믿으면 안 된다. AI 는 코드를 빠르게 만들어 주는 도구이고, 최종 확인은 개발자가 해야 한다.

구분	핵심 내용
회원 인증	Firebase Authentication 으로 회원 가입, 로그인, 로그아웃 구현
데이터 저장	MongoDB Atlas 에 차량, 사용자, 상담 메시지 저장
차량 서비스	검색, 상세 조회, 사진 업로드, 딜러 상담 구현
실시간 기능	Socket.io 로 사용자와 딜러 상담 구현
배포	React 와 Express 서버를 Render 에 배포
AI 확장	상담 로직을 나중에 AI Agent 로 교체할 수 있게 분리

2. 전체 작업 순서

- 1 단계. 프로젝트 구조 확인
- 2 단계. Firebase 회원 가입과 로그인 구현
- 3 단계. MongoDB Atlas 연동
- 4 단계. 차량 등록과 사진 업로드 구현
- 5 단계. 차량 검색 기능 구현
- 6 단계. 차량 상세 화면 구현
- 7 단계. Socket.io 실시간 상담 구현
- 8 단계. AI Agent 확장 구조 정리
- 9 단계. Render 배포
- 10 단계. README 정리

한 번에 전체 기능을 요청하지 않는다. AI 에게는 반드시 작은 단위로 나누어 요청한다.

3. AI 에게 요청하기 전 준비할 내용

3.1 현재 프로젝트 설명

현재 프로젝트는 React + Vite 프론트엔드와 Node.js Express 백엔드로 구성되어 있다.
개발 환경은 GitHub Codespaces 이다.
스타일은 Tailwind CSS 를 사용한다.
데이터베이스는 MongoDB Atlas 를 사용한다.
회원 인증은 Firebase Authentication 을 사용한다.
실시간 상담은 Socket.io 로 구현한다.
배포는 Render 에 할 예정이다.

3.2 기존 폴더 구조 확인

```
ls
find . -maxdepth 2 -type f
```

다음은 내 프로젝트 폴더 구조야.
이 구조를 기준으로 Firebase Auth, MongoDB Atlas, Socket.io 를 단계적으로 붙이고 싶어.
먼저 어떤 파일을 수정해야 하는지 작업 순서만 정리해줘.

[폴더 구조 붙여넣기]

4. Firebase 회원 인증 구현

사용자가 이메일과 비밀번호로 회원 가입하고 로그인할 수 있게 한다. 회원 유형은 일반 사용자와 딜러로 나눈다.

4.1 AI 요청 프롬프트

React + Vite 프로젝트에 Firebase Authentication 을 붙이고 싶어.

요구사항은 다음과 같아.

1. 이메일/비밀번호 회원 가입
2. 이메일/비밀번호 로그인
3. 로그인 상태 유지
4. 로그아웃
5. 회원 가입 시 사용자 유형 선택: 일반 사용자 또는 딜러
6. 로그인한 사용자 정보는 전역에서 사용할 수 있게 구성

현재 프로젝트는 React + Vite + Tailwind CSS 를 사용하고 있어.

필요한 설치 명령어, .env 설정, firebase.js 파일 구성, Login.jsx, Register.jsx 예시를 단계별로 만들어줘.

4.2 확인할 것

- 회원 가입이 되는가?
- 로그인이 되는가?
- 새로고침 후에도 로그인 상태가 유지되는가?
- 로그아웃이 되는가?
- Firebase 콘솔에 사용자가 등록되는가?

5. MongoDB Atlas 연동

차량 정보, 사용자 정보, 상담방, 상담 메시지를 MongoDB Atlas 에 저장한다.

5.1 AI 요청 프롬프트

Express 서버에서 MongoDB Atlas 를 MongoClient 로 연결하고 싶어.

요구사항은 다음과 같아.

1. .env 의 MONGODB_URI 를 사용한다.
2. DB 이름은 car_market 으로 한다.
3. cars, users, chat_rooms, messages 컬렉션을 사용할 예정이다.
4. MongoClient 연결 코드는 별도 파일로 분리한다.
5. 서버 시작 시 MongoDB 연결 여부를 콘솔에 출력한다.
6. 연결 실패 시 원인을 확인할 수 있게 에러 메시지를 출력한다.

server 폴더 기준으로 필요한 파일 구조와 코드를 만들어줘.

5.2 환경 변수 예시

```
PORT=3000
MONGODB_URI=본인의 MongoDB Atlas 접속 문자열
DB_NAME=car_market
CLIENT_URL=http://localhost:5173
```

5.3 확인할 것

- 서버 실행 시 MongoDB 연결 성공 메시지가 나오는가?
- 접속 문자열을 코드에 직접 쓰지 않았는가?
- .env 파일이 GitHub 에 올라가지 않도록 .gitignore 에 포함했는가?

6. 차량 등록과 사진 업로드

딜러가 차량 정보를 등록하고 사진을 함께 업로드할 수 있게 한다. MongoDB 에는 이미지 파일이 아니라 이미지 경로를 저장한다.

6.1 차량 데이터 구조

```
{
  "name": "Sonata Hybrid",
  "company": "HYUNDAI",
  "price": 2800,
  "year": 2023,
  "type": "sedan",
  "fuel": "hybrid",
  "mileage": 35000,
  "location": "서울",
  "description": "출퇴근용으로 적합한 하이브리드 세단",
  "imageUrl": "/uploads/sonata.jpg",
  "dealerId": "firebase-user-uid",
  "dealerName": "김딜러"
}
```

6.2 AI 요청 프롬프트

Express 서버에 차량 등록 API 를 만들고 싶어.

요구사항은 다음과 같아.

1. POST /api/cars
2. multipart/form-data 방식으로 차량 정보와 사진을 함께 받는다.
3. 사진 업로드는 multer 를 사용한다.
4. 업로드된 사진은 uploads 폴더에 저장한다.
5. MongoDB cars 컬렉션에는 차량 정보와 imageUrl 을 저장한다.
6. imageUrl 은 React 에서 바로 보여줄 수 있는 경로로 저장한다.
7. createdAt, updatedAt 도 함께 저장한다.

필요한 설치 명령어, Express 설정, route 코드, React 등록 폼 예시를 만들어줘.

6.3 확인할 것

- 사진 없이 등록했을 때 오류가 나지 않는가?
- 사진을 등록하면 uploads 폴더에 파일이 생기는가?
- MongoDB 에 imageUrl 이 저장되는가?
- React 차량 카드에 사진이 보이는가?

7. 차량 검색 기능

검색 조건은 차량명, 제조사, 가격 범위, 연식 범위로 나눈다. 여러 조건을 함께 적용할 수 있어야 한다.

검색 구분	Query 이름	설명
차량명 검색	keyword	차량명 일부 검색
제조사 검색	company	HYUNDAI, KIA, BMW 등
가격 범위 검색	minPrice, maxPrice	최소 가격과 최대 가격
연식 범위 검색	minYear, maxYear	최소 연식과 최대 연식

7.1 AI 요청 프롬프트

MongoDB cars 컬렉션을 기준으로 차량 검색 API 를 만들고 싶어.

검색 조건은 다음과 같아.

1. keyword: 차량명 검색
2. company: 제조사 검색
3. minPrice, maxPrice: 가격 범위 검색
4. minYear, maxYear: 연식 범위 검색

하나의 API 에서 복합 검색이 가능해야 해.

API 는 다음 주소로 만들고 싶어.

GET /api/cars/search

예:

GET /api/cars/search?keyword=sonata&company=HYUNDAI&minPrice=1000&maxPrice=3000&minYear=2020

MongoDB 쿼리를 동적으로 구성하는 Express 코드를 만들어줘.

React 검색 폼과 검색 결과 출력 예시도 함께 만들어줘.

7.2 확인할 것

- 차량명 일부만 입력해도 검색되는가?
- 제조사만 선택해도 검색되는가?
- 가격 최소값과 최대값이 적용되는가?
- 연식 범위가 적용되는가?
- 여러 조건을 함께 넣어도 정상 검색되는가?

8. 차량 상세 화면

차량을 클릭하면 상세 정보를 확인하고 상담을 시작할 수 있게 한다.

8.1 AI 요청 프롬프트

React 에서 차량 상세 화면을 만들고 싶어.

요구사항은 다음과 같아.

1. URL 은 /cars/:id 형태로 사용한다.
2. Express API GET /api/cars/:id 에서 차량 상세 정보를 가져온다.
3. 차량 사진, 차량명, 제조사, 가격, 연식, 차종, 연료, 주행거리, 지역, 설명, 딜러 이름을 출력한다.
4. 하단에 "딜러와 상담하기" 버튼을 둔다.
5. 버튼을 누르면 상담방을 만들고 채팅 화면으로 이동한다.

React Router 기준으로 필요한 컴포넌트와 API 호출 코드를 만들어줘.

8.2 확인할 것

- 목록에서 상세 화면으로 이동되는가?
- 새로고침해도 상세 정보가 유지되는가?
- 잘못된 차량 ID 로 접근했을 때 오류 화면이 나오는가?
- 상담하기 버튼이 동작하는가?

9. Socket.io 실시간 상담

사용자와 딜러가 차량 상세 화면에서 실시간으로 상담할 수 있게 한다. 메시지는 MongoDB 에 저장한다.

9.1 상담 시나리오

1. 사용자가 차량 상세 화면에서 상담하기 버튼을 누른다.
2. 서버는 차량 ID, 사용자 ID, 딜러 ID 를 기준으로 상담방을 만든다.
3. 사용자는 상담방에 입장한다.
4. 메시지를 보내면 상대방 화면에 실시간으로 표시된다.
5. 메시지는 MongoDB messages 컬렉션에 저장된다.
6. 나중에 다시 들어와도 이전 메시지를 볼 수 있다.

9.2 AI 요청 프롬프트

Express 서버와 React 클라이언트에 Socket.io 실시간 채팅 기능을 붙이고 싶어.

요구사항은 다음과 같아.

1. 서버는 Express 와 같은 포트에서 Socket.io 를 실행한다.
2. 사용자는 roomId 기준으로 상담방에 입장한다.
3. 클라이언트가 send-message 이벤트를 보내면 서버는 메시지를 MongoDB messages 컬렉션에 저장한다.
4. 저장 후 같은 방의 사용자에게 receive-message 이벤트를 보낸다.
5. 상담방 입장 이벤트 이름은 join-room 으로 한다.
6. 메시지 전송 이벤트 이름은 send-message 로 한다.
7. 메시지 수신 이벤트 이름은 receive-message 로 한다.
8. React 에서는 채팅 메시지 목록, 입력창, 전송 버튼을 만든다.

서버 코드와 React ChatRoom.jsx 예시를 단계별로 만들어줘.

9.3 확인할 것

- 브라우저 두 개를 열고 메시지가 실시간으로 오가는가?
- 상담방이 다르면 메시지가 섞이지 않는가?
- 메시지가 MongoDB 에 저장되는가?
- 새로고침 후 이전 메시지를 다시 불러오는가?

10. AI Agent 확장 준비

이번 과제에서 AI Agent 를 완성하지 않아도 된다. 다만 상담 구조는 나중에 AI Agent 가 끼어들 수 있게 만들어 둔다.

10.1 AI 요청 프롬프트

현재 Socket.io 채팅 기능이 있어.

이 기능을 나중에 AI Agent 상담으로 확장하고 싶어.

요구사항은 다음과 같아.

1. 사용자가 보낸 메시지를 `handleChatMessage` 함수에서 처리한다.
2. 현재는 메시지를 딜러에게 전달한다.
3. 나중에는 딜러가 오프라인이면 AI Agent 가 답변할 수 있게 구조를 분리한다.
4. AI Agent 가 참고할 수 있도록 차량 정보, 사용자 질문, 이전 메시지를 함께 가져오는 구조를 만든다.
5. 지금은 실제 OpenAI API 호출은 하지 않는다.
6. 대신 `generateAgentReply` 라는 빈 함수를 만들어 두고, 나중에 교체할 수 있게 한다.

기존 Socket.io 코드에 이 구조를 반영하는 예시를 만들어줘.

10.2 코드 구조 예시

```
async function handleChatMessage({ roomId, senderId, message }) {  
  // 1. 메시지 저장  
  // 2. 상대방 정보 조회  
  // 3. 차량 정보 조회  
  // 4. 딜러 온라인 여부 확인  
  // 5. 현재는 딜러에게 전달  
  // 6. 이후 AI Agent 응답으로 확장  
}  
  
async function generateAgentReply({ car, messages, userMessage }) {  
  // 다음 수업에서 OpenAI API 또는 온디바이스 AI Agent 로 교체  
}
```

11. Render 배포

Express 서버는 Render Web Service 로, React 앱은 Render Static Site 로 배포한다.

11.1 AI 요청 프롬프트

React + Vite 프론트엔드와 Express 백엔드를 Render 에 배포하려고 해.

현재 구조는 다음과 같아.

client: React + Vite

server: Express + MongoDB Atlas + Socket.io

요구사항은 다음과 같아.

1. server 는 Render Web Service 로 배포한다.
2. client 는 Render Static Site 로 배포한다.
3. server 의 환경 변수에는 MONGODB_URI, DB_NAME, CLIENT_URL 을 등록한다.
4. client 의 환경 변수에는 VITE_API_BASE_URL 을 등록한다.
5. Socket.io 도 Render 서버 주소로 연결한다.
6. CORS 설정도 배포 주소 기준으로 수정한다.

Render 배포 절차와 필요한 설정값을 단계별로 정리해줘.

11.2 확인할 것

- Render 서버 주소로 API 호출이 되는가?
- React 배포 주소에서 로그인과 검색이 되는가?
- Socket.io 채팅이 배포 환경에서도 동작하는가?
- 환경 변수가 GitHub 에 노출되지 않았는가?

12. 오류가 났을 때 AI 에게 묻는 방법

오류가 나면 “안 돼요”라고만 묻지 않는다. 아래 네 가지를 함께 붙여서 질문한다.

1. 내가 하려던 작업
2. 실행한 명령어
3. 오류 메시지 전체
4. 관련 코드 일부

12.1 질문 예시

MongoDB Atlas 에 연결하려고 했는데 서버 실행 시 오류가 납니다.

실행한 명령어:
`npm run dev`

오류 메시지:
[오류 메시지 전체 붙여넣기]

관련 코드:
[db.js 또는 server.js 코드 붙여넣기]

.env 설정에서 MONGODB_URI 는 등록했습니다.
이 오류의 원인과 수정 방법을 단계별로 알려주세요.

13. AI 가 만든 코드 검토 기준

- 내가 요청한 기능만 구현했는가?
- 기존 파일 구조를 크게 망가뜨리지 않았는가?
- 환경 변수를 코드에 직접 쓰지 않았는가?
- import 경로가 맞는가?
- 서버와 클라이언트 주소가 맞는가?
- MongoDB 컬렉션 이름이 일관되는가?
- Firebase uid 를 사용자 식별값으로 사용하고 있는가?
- Socket.io 이벤트 이름이 서버와 클라이언트에서 같은가?
- 실행 후 콘솔 오류가 없는가?
- 브라우저 새로고침 후에도 주요 기능이 유지되는가?

14. 주말 작업 추천 순서

시간	작업
1 일 차 오전	Firebase 회원 가입 / 로그인
1 일 차 오후	MongoDB Atlas 연동
1 일 차 저녁	차량 등록 + 사진 업로드
2 일 차 오전	차량 검색 + 차량 상세 화면
2 일 차 오후	Socket.io 상담
2 일 차 저녁	Render 배포 + README 정리

막히는 부분이 있으면 전체를 다시 만들려고 하지 말고, 해당 기능만 분리해서 AI 에게 질문한다.

15. README 에 반드시 적을 내용

프로젝트 소개
사용 기술
실행 방법
환경 변수 설정
Firebase Authentication 설정
MongoDB Atlas 설정
차량 검색 기능
사진 업로드 기능
Socket.io 상담 기능
Render 배포 주소
AI Agent 확장 계획

16. 최종 점검

- 회원 가입이 되는가?
- 로그인이 되는가?
- 차량 등록이 되는가?
- 사진이 보이는가?
- 차량명 검색이 되는가?
- 제조사 검색이 되는가?
- 가격 범위 검색이 되는가?
- 연식 검색이 되는가?
- 차량 상세 화면이 열리는가?
- 상담 메시지가 실시간으로 오가는가?
- 상담 메시지가 MongoDB 에 저장되는가?
- Render 배포 주소에서 접속되는가?
- README 가 작성되어 있는가?

이번 과제의 핵심은 완벽한 서비스를 만드는 것이 아니다. 기존 CRUD 앱을 실제 서비스 구조로 확장해 보는 것이다. AI 는 코드를 대신 작성해 줄 수 있지만, 기능의 흐름을 이해하고 검증하는 일은 개발자의 몫이다.